

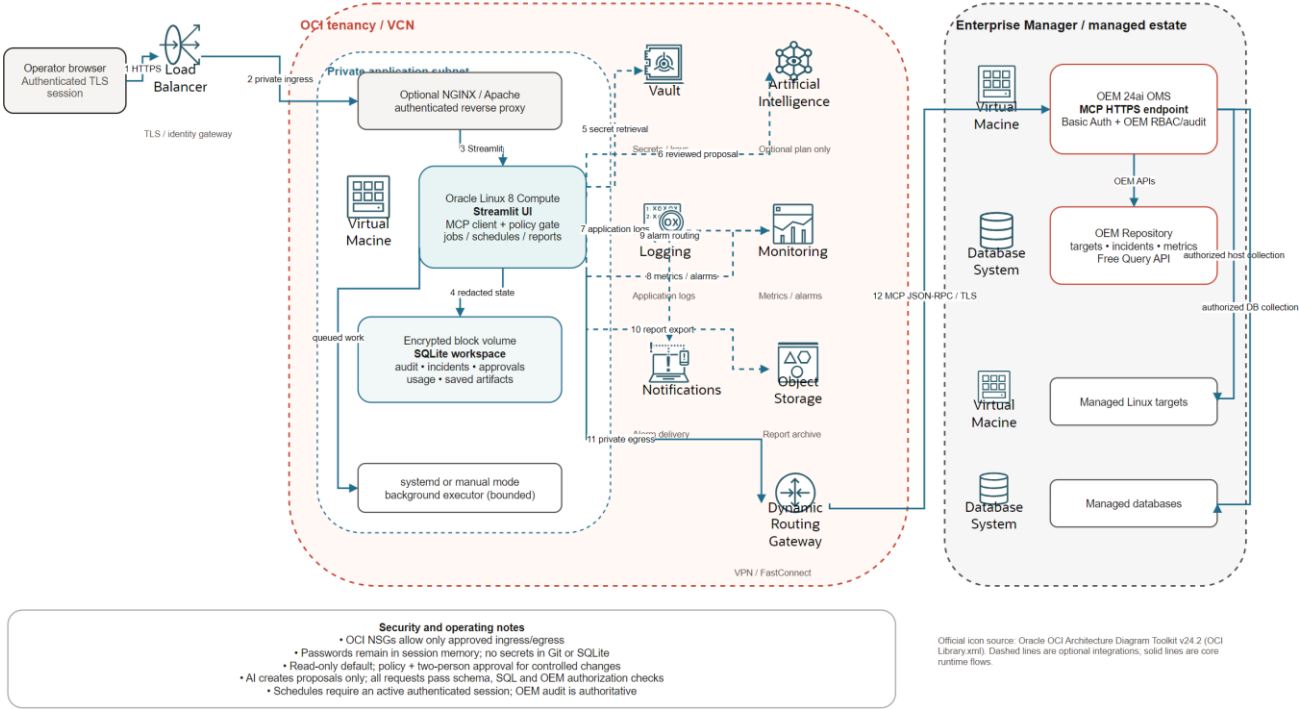
SOFTWARE SOLUTION DESIGN

OEM MCP Streamlit Operations Client

OCI deployment architecture and operator workspace design

OEM MCP Streamlit Operations Client — OCI deployment architecture

Production reference: secure ingress, private application tier, OEM hybrid connectivity, OCI observability and optional AI planning



Reference architecture rendered from the editable draw.io source using Oracle OCI Architecture Diagram Toolkit v24.2 stencils.

Document	Value
Version	1.0.0
Status	Implementation baseline
Date	27 August 2026
Audience	Solution architects, OEM administrators, security reviewers, Linux operators, application maintainers
Scope	Feature priorities 2–13; identity provider integration (priority 1) is out of scope

Document control

Version	Date	Change	Approval state
1.0.0	2026-08-27	Initial architecture and implementation baseline for priorities 2–13	For technical review

Table of contents

Document control 2

Table of contents 2

1. Executive summary 4

2. Objectives, scope and constraints 4

 2.1 Objectives 4

 2.2 In scope 4

 2.3 Out of scope 4

 2.4 Constraints and assumptions 5

3. Architecture 5

 3.1 Logical view 5

 3.2 Runtime flow 5

 3.3 Deployment zones 6

4. Component design 6

5. Feature implementation traceability 7

 5.1 Priority 2 — Live operations dashboard 8

 5.2 Priority 3 — Saved dashboards and runbooks 8

 5.3 Priority 4 — OEM topology explorer 8

 5.4 Priority 5 — Read-only SQL workbench 8

 5.5 Priority 6 — Incident workspace 8

 5.6 Priority 7 — Natural-language assistant 9

 5.7 Priority 8 — Tool-policy engine 9

 5.8 Priority 9 — Two-person approval 9

 5.9 Priority 10 — Multi-OEM fleet 9

 5.10 Priority 11 — Background jobs 9

 5.11 Priority 12 — Schedules, reports and alerts 10

5.12 Priority 13 — Usage and cost.....	10
6. Data design	10
6.1 Secret handling	11
7. Security design	11
7.1 Trust boundaries	11
7.2 Layered execution decision.....	11
8. Interfaces and protocols	12
9. Deployment design	12
9.1 Manual standalone	12
9.2 systemd service.....	12
9.3 OCI reference deployment.....	13
10. Operations and observability.....	13
10.1 Health and diagnostics.....	13
10.2 Backup and recovery.....	13
10.3 Capacity.....	14
11. Failure modes and recovery.....	14
12. Test strategy and acceptance	14
12.1 Automated verification	14
12.2 Acceptance criteria	15
12.3 Required live environment tests.....	15
13. Risks and roadmap.....	15
14. Configuration reference.....	16
15. References	17

Reader note. Headings, tables, captions and page numbers are structured for navigation and review. Update the TOC field after opening the DOCX.

1. Executive summary

The OEM MCP Streamlit Operations Client is a security-conscious Oracle Linux 8 application that connects to an Oracle Enterprise Manager 24ai MCP endpoint, discovers the operations authorized for the supplied OEM account, and exposes those operations through an operator workspace. Version 1.0.0 extends the original connection, discovery, request, metrics, history and diagnostics functions with the twelve requested priorities numbered 2 through 13.

The design uses a synchronous HTTP/JSON-RPC OEM client, a Streamlit presentation tier, local redacted SQLite stores, bounded in-process workers and deny-first policy enforcement. Optional OCI services—Load Balancer, Vault, Generative AI, Logging, Monitoring, Notifications and Object Storage—are shown as deployment integrations, not mandatory runtime dependencies.

- Default posture: HTTPS, TLS verification, loopback listener, read-only tool classification and one-statement SELECT/WITH validation.
- Controlled changes: explicit policy allow rule, administrator mutating-tools gate and an unexpired approval from a different operator.
- AI boundary: deterministic or OCI Generative AI planning proposes one discovered tool; execution remains a separate, policy-checked action.
- Persistence boundary: non-secret profiles, redacted arguments, incidents, approvals, jobs, schedules, dashboards, runbooks and usage records are stored locally.
- Authority boundary: OEM determines available tools and target access; the client cannot elevate privileges.

2. Objectives, scope and constraints

2.1 Objectives

- Provide one operator interface for discovery, live observation, analysis, governed execution and evidence capture.
- Avoid hard-coding OEM tool names where server releases and account privileges can change the advertised surface.
- Keep credentials out of Git, profiles, history, application logs and generated reports.
- Support manual standalone and systemd service deployment with configurable listener ports.
- Make the solution testable without a live OEM server through fake transports and pure feature modules.

2.2 In scope

Priorities 2–13, the editable OCI architecture diagram, Linux deployment controls, local data model, testing, operations and documentation are in scope.

2.3 Out of scope

- Priority 1 identity-provider/OIDC implementation. Shared deployments still require an authenticated TLS reverse proxy or equivalent gateway.
- Replacement of OEM authorization, audit, incident management or repository data.
- Long-running distributed job execution across multiple application nodes.
- Automatic persistence of OEM passwords or OCI AI API keys.
- Direct alert delivery code. OCI Notifications or an allow-listed enterprise integration is a platform-layer extension.

2.4 Constraints and assumptions

Constraint / assumption	Design response
OEM 24.1.0.12+ exposes HTTP JSON-RPC MCP	Use initialize, initialized notification, discovery and tools/call over POST.
OEM uses HTTP Basic Authentication	Keep password in process session memory; never persist or log it.
Prompt/resource lists may be empty	Discover and display all capability classes but center workflows on tools.
Python 3.9–3.13 supported	Pin Streamlit 1.55+ and test on Python 3.11 in CI.
Single Streamlit process	Use bounded in-process workers and SQLite WAL; document scale-out limitations.

3. Architecture

3.1 Logical view

OEM MCP Streamlit Operations Client — OCI deployment architecture

Production reference: secure ingress, private application tier, OEM hybrid connectivity, OCI observability and optional AI planning

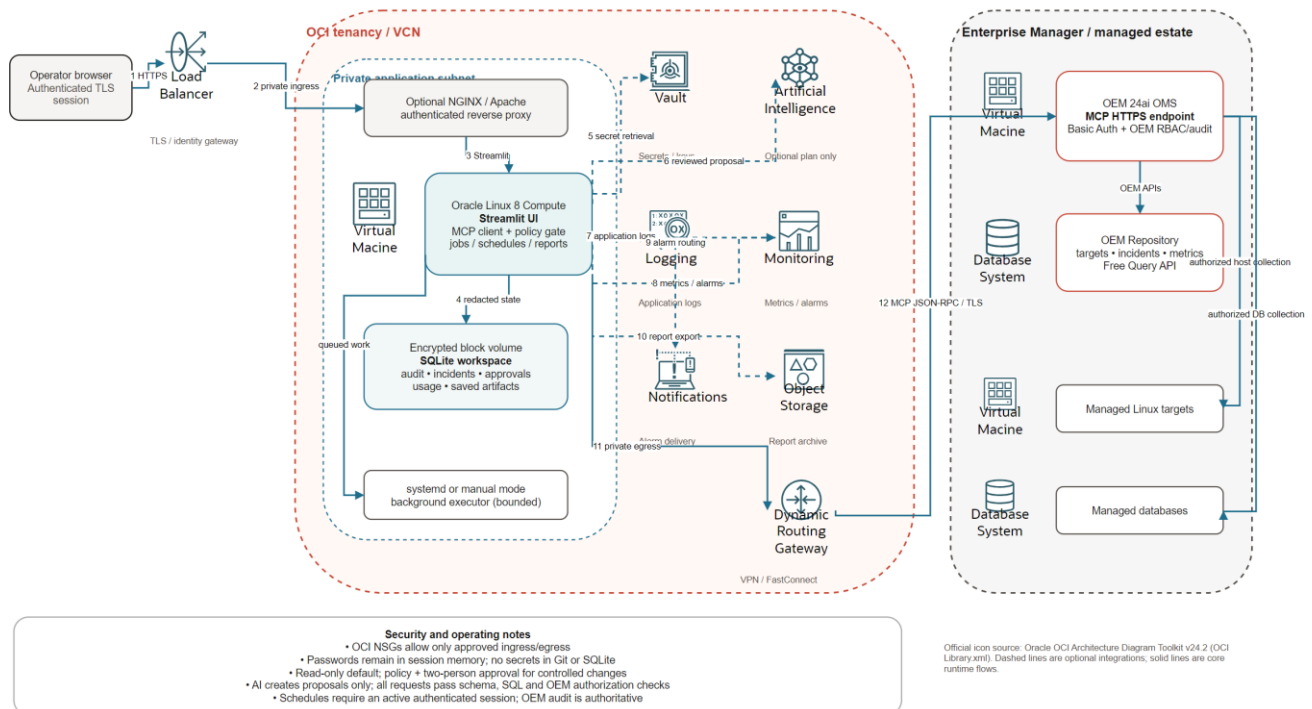


Figure 1 — OCI deployment and OEM hybrid connectivity reference architecture.

3.2 Runtime flow

1. The operator reaches Streamlit through an authenticated TLS ingress. Loopback remains the application default.
2. The client builds a validated connection profile and retains the OEM password only in Streamlit session memory.

3. MCP initialize negotiates a supported protocol version; discovery retrieves the account-authorized server surface.
4. Every invocation passes policy, approval and tool/SQL safety gates before tools/call reaches OEM.
5. Redacted local events and workspace state are committed to SQLite; transport logs exclude request bodies and credentials.
6. Optional OCI services provide secret retrieval, AI planning, centralized observability, alarm delivery and report archival.

3.3 Deployment zones

Zone	Components	Required controls
Ingress	OCI Load Balancer or enterprise reverse proxy	TLS, user authentication, request-size limits, restricted source networks
Private application subnet	Oracle Linux 8, Streamlit, worker, SQLite/block volume	NSG, host hardening, least privilege, backups, egress allow-list
OCI service plane	Vault, AI, Logging, Monitoring, Notifications, Object Storage	IAM policies, private endpoints where supported, encryption, retention
OEM / managed estate	OMS MCP endpoint, repository, Linux and database targets	OEM RBAC, TLS, audit, approved network path

4. Component design

Component	Responsibility	State / trust notes
Streamlit application	Operator workflows, form generation, results, topology and dashboards	Session state is process-local; shared access requires upstream identity.
OemMcpClient	MCP lifecycle, pagination, session headers, JSON-RPC validation and tool calls	Uses OEM Basic Auth and verified HTTPS.
PolicyEngine	Ordered deny-first role/tool/target rules	Built-in fallback permits recognized read-only tools and validated ExecuteSql.
WorkspaceStore	Artifacts, incidents, approvals, jobs, schedules, alerts and usage	SQLite WAL; secret-key fields are redacted and payloads are capped.
Background job manager	Bounded ThreadPoolExecutor and durable state transitions	In-process only; queued/running state requires reconciliation after restart.
Assistant planner	Local lexical ranking or OCI AI OpenAI-compatible proposal	Proposes one discovered tool; never calls OEM directly.

Component	Responsibility	State / trust notes
Topology inference	Extract common target/host/database fields and produce Graphviz DOT	Best effort over live structured content.
History/logging	Connection/execution timelines and rotating operational log	Sanitized endpoint, username fingerprint, no request bodies.

5. Feature implementation traceability

Priority	Capability	Implemented design	GUI location
2	Live operations dashboard	Five-second local fragment; OEM metric candidate execution; chart/table; saved snapshot	Operations
3	Saved dashboards and runbooks	SQLite artifacts with JSON payloads; runbook checklist and dashboard inspection	Workspace
4	OEM topology explorer	Association tool execution, generic record extraction, node/edge inference and Graphviz	Topology
5	Read-only SQL workbench	ExecuteSql discovery, SELECT/WITH validation, automatic FETCH FIRST cap and query library	SQL
6	Incident investigation workspace	Severity/status, append-only timeline events, result correlation and resolution state	Incidents
7	Natural-language assistant	Deterministic ranking and optional OCI AI proposal with explicit reviewed execution	Assistant
8	Tool-policy engine	JSON role/tool/target patterns, deny precedence, safe fallback and decision explanation	Governance
9	Approval workflow	Request hash, expiry, distinct approver, decision note and execution-time approval lookup	Governance
10	Multi-OEM support	Simultaneous in-memory profile clients and cross-server capability matrix	Workspace / Multi-OEM
11	Background job manager	Bounded worker pool, durable queued/running/success/failure state and report download	Automation / Jobs
12	Alerting and scheduled reports	Persistent intervals, due-job queuing, downloadable JSON reports and local threshold alerts	Automation
13	Usage and cost accounting	MCP/AI event ledger, token counts, configurable price estimates, latency and CSV export	Usage & cost

5.1 Priority 2 — Live operations dashboard

Local Linux health uses CPU, memory and disk measurements to calculate an indicative score. A Streamlit fragment refreshes every five seconds. Discovered host/database candidates can be called, charted and saved as a dashboard snapshot.

Control. The OEM call follows the standard policy and safety path; no hidden polling occurs without the tab being rendered.

5.2 Priority 3 — Saved dashboards and runbooks

Artifacts use kind/name uniqueness and JSON payloads. Dashboards preserve the tool, arguments and redacted result snapshot. Runbooks preserve purpose and ordered operator steps.

Control. Artifacts are local working aids, not authoritative OEM records. Secret-key fields are redacted before serialization.

5.3 Priority 4 — OEM topology explorer

The explorer ranks live association tools, executes a reviewed request and infers target nodes and host/database or parent/child edges from common field names.

Control. Inference is best effort. Operators must compare the graph with OEM target topology before change decisions.

5.4 Priority 5 — Read-only SQL workbench

The workbench uses only the discovered ExecuteSql operation. It accepts a single SELECT or WITH statement and appends FETCH FIRST n ROWS ONLY when no row bound is present.

Control. DDL, DML, PL/SQL, stacked statements and mutation keywords are rejected by default.

5.5 Priority 6 — Incident workspace

Incidents contain title, severity, status, summary and a timestamped actor/event/note timeline. The last result can be filtered by investigation keywords and referenced in notes.

Control. Local incidents support triage; OEM remains the authoritative incident and event store.

5.6 Priority 7 — Natural-language assistant

The local planner ranks only discovered tools. The optional OCI AI planner sends a compact discovered schema list and requires strict JSON containing one known tool and arguments.

Control. The assistant does not execute automatically. Proposals pass JSON Schema, policy, approval and OEM authorization at execution.

5.7 Priority 8 — Tool-policy engine

Ordered JSON rules match operator role, tool glob and extracted target identifiers. Any matching deny wins; an allow can require approval.

Control. With no explicit match, recognized read-only tools and validated ExecuteSql are allowed; other tools are denied.

5.8 Priority 9 — Two-person approval

The request hash binds the sanitized endpoint, exact tool and canonical arguments. Secret-valued fields contribute one-way SHA-256 fingerprints while persisted argument values remain redacted. Approvals expire and cannot be decided by the same case-insensitive operator identity.

Control. This control is procedural until priority 1 supplies cryptographically authenticated operator identities.

5.9 Priority 10 — Multi-OEM fleet

Saved non-secret profiles can be connected simultaneously with passwords held only in session memory. Discovery results form a cross-OEM tool availability matrix.

Control. Connections are process-local and must be recreated after restart. Passwords are never written to the profile store.

5.10 Priority 11 — Background jobs

A bounded executor records queued, running and terminal state. Results are redacted and capped; completed report JSON can be downloaded.

Control. The manager is not a distributed queue. After process loss, in-flight records require operator reconciliation.

5.11 Priority 12 — Schedules, reports and alerts

Schedules persist profile, read-only tool, arguments, interval and next run. Due schedules queue reports only while an authenticated active session exists. Local health rules create alert records.

Control. External notification delivery is deliberately delegated to OCI Notifications or an approved enterprise channel.

5.12 Priority 13 — Usage and cost

Every MCP operation records provider, operation, latency, status and zero service cost. AI planning records model token usage and computes a configured price estimate.

Control. Estimates are not invoices. OCI Cost Analysis and service billing remain authoritative.

6. Data design

Store/table	Key fields	Retention / protection
history.connections	time, sanitized endpoint, user fingerprint, protocol, outcome, latency	Local operational history; no username/password
history.executions	time, endpoint, tool, status, latency, redacted arguments	Arguments capped at 4 KB
workspace.artifacts	kind, unique name, description, payload, timestamps	Dashboard/runbook/query working data
workspace.incidents	severity, status, summary, timeline JSON	Local triage evidence; notes capped
workspace.approvals	request hash, requester, approver, status, expiry	Distinct actors; exact request binding
workspace.jobs	kind, state, payload/result, error, timestamps	Redacted/capped data; restart reconciliation
workspace.schedules	profile, tool, args, interval, next run	Read-only scheduling only

Store/table	Key fields	Retention / protection
workspace.usage_events	provider/model/tokens/cost/latency/status	Estimated accounting; CSV export

6.1 Secret handling

- Keys matching password, secret, token, authorization, credential, API key or private key are replaced before persistence.
- OEM passwords and AI keys are never included in profiles, approval hashes, history messages or application logs.
- Runtime environment files are created outside Git and protected by filesystem ownership/mode in deployment scripts.
- SQLite and logs should reside on encrypted storage with host backup and retention policy defined by the operator organization.

7. Security design

7.1 Trust boundaries

Boundary	Threats	Controls
Browser ↔ ingress	Credential theft, session hijack, exposure	TLS, upstream authentication, loopback default, firewall/NSG, secure cookies at proxy
App ↔ OEM	MITM, privilege misuse, injection	TLS verification/private CA, OEM Basic Auth/RBAC, schema validation, policy, SQL filter
App ↔ OCI AI	Prompt/data disclosure, invented tools	Opt-in configuration, compact schema, HTTPS, known-tool validation, proposal-only workflow
App ↔ local data	Secret leakage, tampering, disk loss	Redaction, capped fields, restrictive permissions, encrypted volume, backup/restore tests
Operator ↔ approval	Self-approval, stale approval, request substitution	Distinct operator string, expiry and exact request hash

7.2 Layered execution decision

- Reject if no initialized OEM client exists.
- Evaluate role/tool/target policy; matching denies take precedence.
- If required, verify an approved, unexpired hash for the exact endpoint/tool/arguments.
- Validate read-only classification or administrator mutating gate.
- For ExecuteSql, validate one statement and read-only semantics unless the separate non-select gate is explicitly enabled.

- Invoke OEM and rely on OEM authorization/validation; record redacted outcome and usage.

Identity limitation. Operator and approver strings are not strong identities in version 1.0.0 because priority 1 was excluded. Production use of controlled changes requires an authenticated reverse proxy and a future trusted-header/OIDC binding design.

8. Interfaces and protocols

Interface	Protocol / format	Behavior
OEM MCP	HTTPS POST, JSON-RPC 2.0, MCP 2025-11-25 or 2025-06-18	initialize, initialized notification, paginated discovery, tools/call, ping, session close
Streamlit	HTTP behind authenticated TLS ingress	Configurable address/port; 127.0.0.1 default
SQLite	Local file / WAL	History and workspace databases under OEM_MCP_DATA_DIR
OCI Generative AI	OpenAI-compatible HTTPS chat/completions	Optional proposal-only planner; bearer key supported by app configuration
Graphviz	DOT rendered by Streamlit	Inferred topology; no external graph persistence
Exports	JSON and CSV	Redacted job/history/diagnostic/usage downloads

9. Deployment design

9.1 Manual standalone

- Run scripts/install-manual.sh with Python 3.11 to build .venv and create .runtime/oem-mcp-streamlit.env mode 0600.
- Start with scripts/start-standalone.sh and explicit --env-file, --address and --port; stop/status use the matching port-scoped PID file.
- Use this mode for controlled operator sessions, testing and troubleshooting—not for unattended shared service unless supervised externally.

9.2 systemd service

- Run scripts/install-systemd.sh as root with service user/group, Python binary, address and port.
- The installer creates /etc/oem-mcp-streamlit/oem-mcp-streamlit.env, /opt/oem-mcp-streamlit/venv, /var/lib/oem-mcp-streamlit and /var/log/oem-mcp-streamlit.
- Start, stop, restart and inspect using the supplied service scripts and journalctl; smoke-test the configured listener after every update.

9.3 OCI reference deployment

OCI resource	Use	Status in application
VCN/private subnet/NSG	Application isolation and allowed paths	Deployment requirement
Compute + block volume	Oracle Linux 8 runtime and durable local stores	Deployment requirement
Load Balancer	TLS ingress and upstream authentication integration	Optional platform integration
Vault	Secret/key governance	Optional; app currently consumes environment/session values
Generative AI	Optional natural-language tool proposal	Implemented OpenAI-compatible adapter; opt-in
Logging/Monitoring	Central logs, metrics and alarms	Optional platform agents/integration
Notifications	Alarm delivery	Optional platform integration
Object Storage	Report/archive retention	Optional platform integration
DRG/VPN/FastConnect	Private path to OEM / managed estate	Environment-specific network integration

10. Operations and observability

10.1 Health and diagnostics

- Use Diagnostics for safe endpoint, protocol, capability count, paths and feature-gate state.
- Use the local operations fragment for CPU, load, memory, disk and indicative health score.
- Use scripts/diagnose.sh for local prerequisites and optional live OEM initialize/discovery.
- Use scripts/smoke-test.sh and the Streamlit health endpoint after start/redeploy.
- Use History & logs for sanitized connection and execution timelines; use OEM audit for authoritative action records.

10.2 Backup and recovery

- Back up data/history.sqlite3, data/workspace.sqlite3 and non-secret profile JSON using an application-consistent method.
- Protect runtime environment files separately as secrets; do not place them in application backups that have broader readers.
- Test restore on a non-production host and confirm schema initialization is idempotent.
- Treat queued/running jobs at crash time as unknown and reconcile them with OEM audit before rerun.

10.3 Capacity

The design targets a small operator group and one application process. SQLite WAL and two worker threads are the default. Increase OEM_MCP_JOB_WORKERS only after verifying OEM request limits, host resources and request thread safety. For horizontal scale, replace local session/workspace/job state with shared identity, database and queue services.

11. Failure modes and recovery

Failure	Detection	Recovery / safety outcome
OEM unreachable / TLS failure	Connection error, diagnostic, transport log	No execution; correct route/CA and reconnect
MCP session expires	404 with session ID	Reconnect and rediscover before retry
Policy file invalid	Startup governance warning	Built-in read-only fallback remains active
Approval expired	Execution lookup fails	Create a new exact request and independent decision
Worker exception	Job failed with exception type	Inspect OEM audit/log, correct input, deliberately resubmit
Process restart	In-memory clients/jobs lost	Reconnect fleet; reconcile queued/running persisted jobs
SQLite unavailable	Initialization/write error	Stop service, restore permissions/storage, integrity-check and restore
AI response invalid	No choice, invalid JSON/tool/arguments	Reject proposal; use local planner or manual request
Topology fields unknown	Zero inferred nodes	Inspect raw result and use OEM topology directly

12. Test strategy and acceptance

12.1 Automated verification

- Compile app, modules and tests on Python 3.11.
- Ruff lint all Python and ShellCheck all installation/runtime scripts.
- Run pytest with coverage over transport, configuration, safety, history and all priority 2–13 feature modules.
- Run pip-audit against direct runtime dependencies.
- Run Streamlit AppTest and assert every top-level feature tab renders without an exception.
- Validate release metadata and create one commit-derived GitHub release after successful tests on main.

12.2 Acceptance criteria

ID	Criterion	Evidence
AC-01	Connect, initialize and discover using a permitted OEM account	Live diagnostic and OEM audit
AC-02	No credential is found in Git diff, profile, SQLite test data or logs	Secret scan and redaction tests
AC-03	Every priority 2–13 has a visible workflow and persistent or computed output	AppTest plus feature traceability table
AC-04	Mutation is denied unless policy, approval and admin gate all permit it	Policy/approval unit tests and controlled test
AC-05	SQL mutation and unbounded output are controlled	SQL tests and generated FETCH FIRST clause
AC-06	Manual and systemd modes start, health-check, stop and update on configured ports	Oracle Linux deployment smoke test
AC-07	draw.io, SVG, PNG, DOCX and PDF architecture/design artifacts render correctly	Visual QA and repository files

12.3 Required live environment tests

- Connect to each intended OEM profile and compare the discovered capability matrix against expected RBAC.
- Execute representative host, database, topology and SELECT queries against non-production targets.
- Verify upstream authentication, proxy timeouts, request-size limits, TLS chain and logout/session behavior.
- Stop the application during an active background job, restart and exercise the reconciliation procedure.
- Verify backup/restore of both SQLite files and confirm runtime secrets are excluded from general backups.

13. Risks and roadmap

Risk	Current mitigation	Recommended next step
Operator identity is self-declared	Reverse-proxy requirement and distinct string check	Implement priority 1 OIDC/trusted identity binding
In-process jobs are not durable execution	Persist status and require reconciliation	Use OCI Queue/Streaming or an enterprise task queue
SQLite limits horizontal scale	Single-node deployment and WAL	Move workspace/audit state to managed database
AI key adapter is not IAM-signed	Opt-in bearer key and no persistence	Add OCI SDK resource-principal/instance-principal signing

Risk	Current mitigation	Recommended next step
Generic topology inference can miss OEM fields	Raw result remains available	Add OEM-version-specific schema adapters
External alert delivery is platform-owned	Persistent local alert records	Integrate OCI Notifications with approved IAM policy
Cost is estimated	Configurable rates and clear label	Reconcile with OCI Cost Analysis / usage reports

14. Configuration reference

Variable	Purpose	Safe default / note
STREAMLIT_ADDRESS / PORT	Listener	127.0.0.1 / 8501
OEM_MCP_ENDPOINT	OEM MCP URL	HTTPS endpoint required
OEM_MCP_USERNAME / PASSWORD	OEM credentials	Leave password blank and enter in GUI
OEM_MCP_VERIFY_TLS / CA_BUNDLE	TLS trust	true / optional private CA
OEM_MCP_POLICY_FILE	Tool/target policy	config/policy.example.json
OEM_MCP_OPERATOR_ID / ROLE	Procedural identity and policy role	operator / operator
OEM_MCP_ALLOW_MUTATING_TOOLS	Administrative mutation gate	false
OEM_MCP_ALLOW_NONSELECT_SQL	Administrative SQL gate	false
OEM_MCP_JOB_WORKERS	Bounded executor size	2
OCI_GENAI_OPENAI_ENDPOINT	Optional OCI AI base URL	Region-specific HTTPS URL
OCI_GENAI_MODEL / API_KEY	Optional model and credential	Blank; never commit a real key
OCI_GENAI_*_USD_PER_MILLION	Cost estimate rates	0 until current rates are configured

15. References

- Oracle Enterprise Manager 24ai — Enterprise Manager Model Context Protocol Server: <https://docs.oracle.com/en/enterprise-manager/cloud-control/enterprise-manager-cloud-control/24.1/emadm/enterprise-manager-model-context-protocol-server.html>
- Oracle blog — Oracle Enterprise Manager MCP Server: Bringing Enterprise Manager Data to AI Agents: <https://blogs.oracle.com/observability/oracle-enterprise-manager-mcp-server-bringing-enterprise-manager-data-to-ai-agents>
- Oracle OCI Architecture Diagram Toolkit and graphics guidance: <https://docs.oracle.com/en-us/iaas/Content/General/Reference/graphicsfordiagrams.htm>
- Oracle OCI Generative AI OpenAI-compatible API: <https://docs.oracle.com/en-us/iaas/Content/generative-ai/openai-compatible-api.htm>
- Oracle Cloud Adoption Framework — Monitoring and Observability: <https://docs.oracle.com/en-us/iaas/Content/cloud-adoption-framework/monitoring-and-observability.htm>
- Model Context Protocol lifecycle specification 2025-11-25: <https://modelcontextprotocol.io/specification/2025-11-25/basic/lifecycle>
- Project README, SECURITY.md, tests and source modules in this repository, version 1.0.0.

Artifact set. The repository folder docs/architecture contains the editable draw.io source, SVG and PNG renders, this DOCX and its PDF rendering. The README embeds the SVG and links all source formats.